



Compact Course Programming. Python

- Python basics -

Prof. Galyna Tabunshchyk

University of Applied Science and Arts Dortmund

galyna.Tabunshchyk@fh-dortmund.de

Lecture Plan

- Python types: tuple, set, dictionaries
- Python functions

Python Tuple operations

Description	Python Expression	Example	Results
Creating a tuple (item1, item2, ...)	# empty tuple (item1,) # Tuple with one item, note comma is required	tuple = ('a','b','c', 'd',1,2,3) tuple = () tuple = (1,) ('a','b','c','d',1,2,3)	() 1
Accessing items in tuple	tuple[index] tuple[start_index: end_index]	tuple = ('a','b','c','d',1,2,3) tuple[2] tuple[0:2]	c a, b, c
Length	len(tuple) 3	len((1, 2, 3))	3
Concatenation	tuple_1 + tuple_2	(1, 2, 3) + (4, 5, 6)	(1, 2, 3, 4, 5, 6)
Repetition	tuple * int	('Hello',) * 4	('Hello', 'Hello', 'Hello', 'Hello')
Membership	item in tuple	3 in (1, 2, 3)	TRUE
Iteration	for x in tuple: print(x)	for x in (1, 2, 3): print (x)	1 2 3

Working with Tuple

```
my_tuple = ()  
print(my_tuple)  
my_tuple = (1, 2, 3)  
print(my_tuple)  
my_tuple = (1, "Hello", 3.4)  
print(my_tuple)  
my_tuple = ("mouse", [8, 4, 6], (1, 2, 3))  
print(my_tuple)
```

```
()  
  
(1, 2, 3)  
  
(1, 'Hello', 3.4)  
  
(  
'mouse', [8, 4, 6], (1, 2, 3))
```

```
my_tuple = 3, 4.6, "dog"  
print(my_tuple)  
a, b, c = my_tuple  
print(a)  
print(b)  
print(c)
```

```
(3, 4.6, 'dog')  
  
3  
4.6  
dog
```

Working with Tuple

```
my_tuple = (4, 2, 3, [6, 5])  
my_tuple[3][0] = 9  
print(my_tuple)
```

(4, 2, 3, [9, 5])

```
my_tuple = ('n', 'e', 'w', 'v', 'a', 'l', 'u',  
'e')  
print(my_tuple)
```

('n', 'e', 'w', 'v', 'a', 'l', 'u', 'e')

```
t1=(1, 2, 3)  
t2=(4, 5, 6)  
print(t1,t2)  
print(t1 + t2)  
print(("Repeat",) * 3)
```

(1, 2, 3) (4, 5, 6)
(1, 2, 3, 4, 5, 6)
('Repeat', 'Repeat', 'Repeat')

Working with Tuple

```
my_tuple = ('a', 'p', 'p', 'l', 'e',)
```

```
print("number of p is",  
my_tuple.count('p'))  
print("index l is", my_tuple.index('l'))
```

```
number of p is 2  
index l is 3
```

```
print('a' in my_tuple)  
print('b' in my_tuple)  
print('g' not in my_tuple)
```

```
True  
False  
True
```

Sets

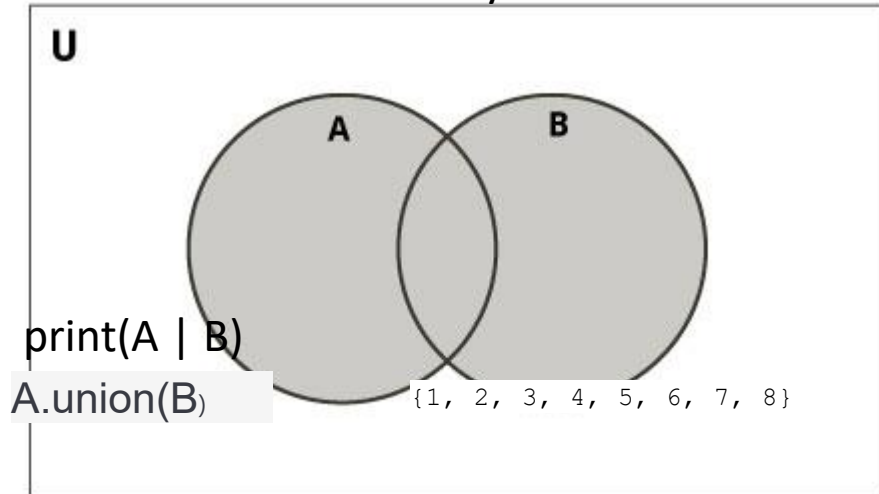
- The collection of items is unordered.
- No duplicate items will be stored, which means that each item is unique.
- Sets are mutable, which means the items of it can be changed

Sets

<pre>my_set = {1, 2, 3} print(my_set) my_set = {1.0, "Hello", (1, 2, 3)} print(my_set)</pre>	<pre>{1, 2, 3} {1.0, 'Hello', (1, 2, 3)}</pre>
<pre>my_set = {1, 3} print(my_set) my_set.add(2) print(my_set) my_set.update([2, 3, 4]) print(my_set) my_set.update([4, 5], {1, 6, 8}) print(my_set)</pre>	<pre>{1, 3} {1, 2, 3} {1, 2, 3, 4} {1, 2, 3, 4, 5, 6, 8}</pre>
<pre>my_set.remove(6) print(my_set) my_set.discard(2) print(my_set)</pre>	<pre>{1, 2, 3, 5, 8} {1, 3, 5, 8}</pre>

Set Operation

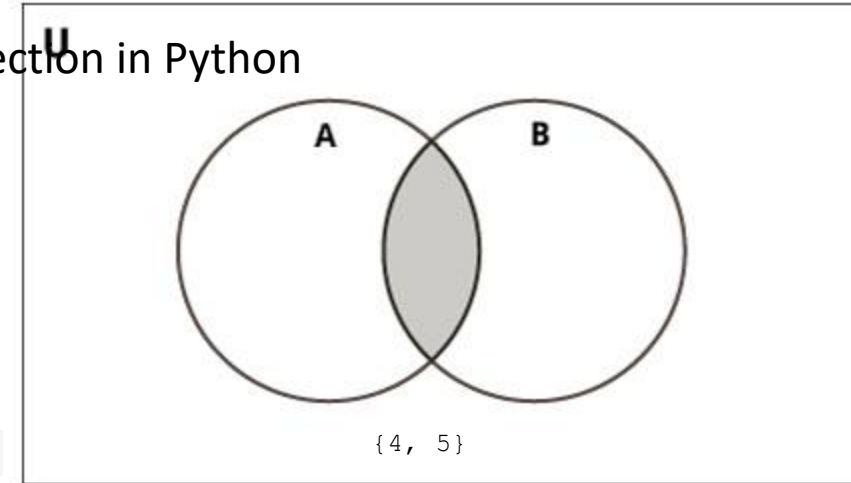
Set Union in Python



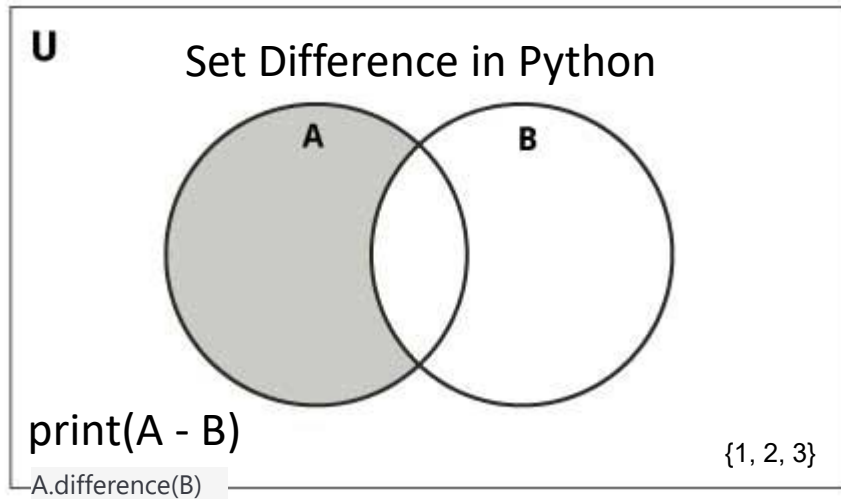
$A = \{1, 2, 3, 4, 5\}$
 $B = \{4, 5, 6, 7, 8\}$

Set Intersection in Python

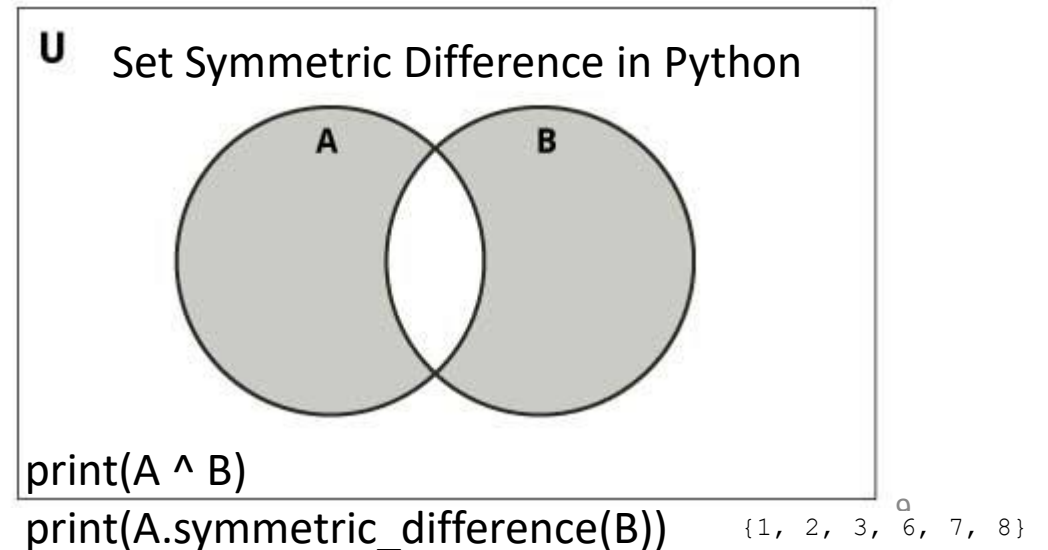
`print(A & B)`
`B.intersection(A)`



Set Difference in Python



Set Symmetric Difference in Python



Dictionaries

```
my_dict = {}  
print(my_dict)
```

```
{}
```

```
my_dict = {1: 'apple', 2: 'ball'}  
print(my_dict)
```

```
{1: 'apple', 2: 'ball'}
```

```
my_dict = {'name': 'John', 1: [2, 4, 3]}  
print(my_dict)
```

```
{'name': 'John', 1: [2, 4, 3]}
```

```
my_dict = dict({1:'apple', 2:'ball'})  
print(my_dict)  
my_dict = dict([(1,'apple'), (2,'ball')])  
print(my_dict)
```

```
{1: 'apple', 2: 'ball'}
```

```
{1: 'apple', 2: 'ball'}
```

Dictionaries

```
tel = {'jack': 4098, 'sape': 4139}
print(tel)
tel['guido'] = 4127
print(tel)
del(tel['sape'])
tel['irv'] = 4127
print(tel)
print(list(tel))
print(sorted(tel))
print('jack' not in tel)
```

```
{'jack': 4098, 'sape': 4139}
```

```
{'jack': 4098, 'sape': 4139, 'guido':  
4127}
```

```
{'jack': 4098, 'guido': 4127, 'irv':  
4127}
```

```
['jack', 'guido', 'irv']
```

```
['guido', 'irv', 'jack']
```

```
False
```

Dictionary Methods

Method	Description
<code>clear()</code>	Removes all items from the dictionary.
<code>copy()</code>	Returns a shallow copy of the dictionary.
<code>fromkeys(seq[, v])</code>	Returns a new dictionary with keys from seq and value equal to v (defaults to None).
<code>get(key[,d])</code>	Returns the value of the key. If the key does not exist, returns d (defaults to None).
<code>items()</code>	Return a new object of the dictionary's items in (key, value) format.
<code>keys()</code>	Returns a new object of the dictionary's keys.
<code>pop(key[,d])</code>	Removes the item with the key and returns its value or d if key is not found. If d is not provided and the key is not found, it raises KeyError.
<code>popitem()</code>	Removes and returns an arbitrary item (key, value). Raises KeyError if the dictionary is empty.
<code>setdefault(key[,d])</code>	Returns the corresponding value if the key is in the dictionary. If not, inserts the key with a value of d and returns d (defaults to None).
<code>update([other])</code>	Updates the dictionary with the key/value pairs from other, overwriting existing keys.
<code>values()</code>	Returns a new object of the dictionary's values

Dictionaries

```
>>> knights = {'gallahad': 'the  
pure', 'robin': 'the brave'}
```

```
>>> for k, v in knights.items():
```

```
...     print(k, v)
```

```
...
```

```
gallahad the pure
```

```
robin the brave
```

List vs. Tuples vs. Set vs. Dictionary

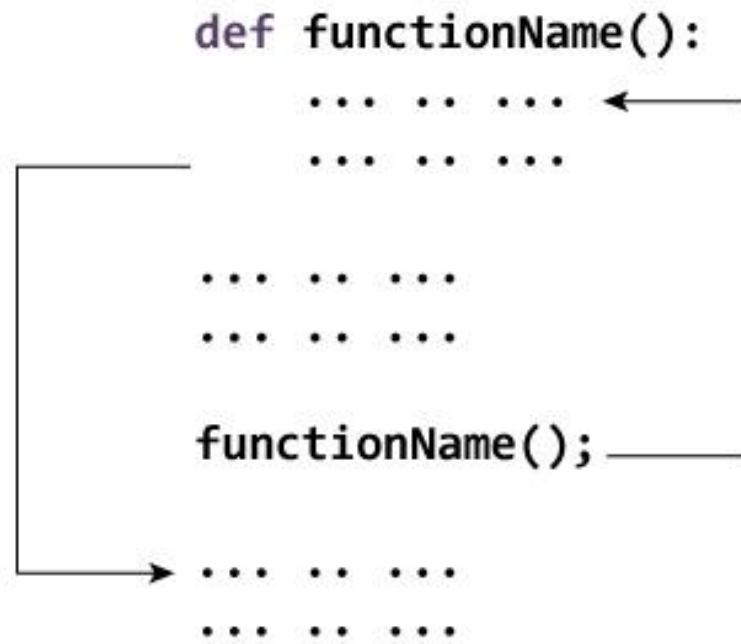
- **List:** Use when you need an ordered sequence of homogenous collections, whose values can be changed later in the program
- **Tuple:** Use when you need an ordered sequence of heterogeneous collections whose values need not be changed later in the program
- **Set:** when you don't need to store duplicates and are not concerned about the order of the items. You just want to know whether a particular value already exists or not
- **Dictionary:** it is ideal for use when you need to relate values with keys, in order to look them up efficiently using a key

Python basics

- function characteristics:
 - has a name
 - has parameters (0 or more)
 - has a docstring (optional but recommended)
 - has a body
 - returns something

Python functions

```
def functionName():  
    ... ..  
    ... ..  
  
    ... ..  
    ... ..  
  
functionName();
```



```
def is_even( i ):  
    """  
    Input: i, a positive int  
    Returns True if i is even, otherwise False  
    """  
    print("inside is_even")  
    return i%2 == 0  
  
is_even(3)
```

keyword

name

parameters or arguments

specification, docstring

body

later in the code, you call the function using its name and values for parameters

Standard function

Pre-defined functions

- `L=[1,2,3]`
- `L1=[4,5,6]`
- `print(len(L))`
- `print(sum(L1))`
- `print(max(L1))`
- `print(min(L))`
- `L1.sort()`
- `print(L1)`

User defined functions

- `def mymax(i,j):`
- `if i>j:`
- `return i`
- `else:`
- `return j`
-
- `i=int(input('i='))`
- `j=int(input('j='))`
- `print(mymax(i,j))`

Global and Local Variables

In Python, a variable declared **outside** of the function or **in global scope** is known as a global variable. This means that a **global variable** can be accessed inside or outside of the function.

A variable declared inside the function's body or **in the local scope** is known as a **local variable**.

A **function** defined inside another **function** is called a **nested function**. **Nested functions** can access variables of the enclosing scope. In **Python**, these non-local variables are read only by default and we must declare them explicitly as non-local (using `nonlocal` keyword) in order to modify them.

map() function

- map() function returns a map object(which is an iterator) of the results after applying the given function to each item of a given iterable (list, tuple etc.)
 - Syntax : map(fun, iter)
 - Parameters :
 - fun : It is a function to which map passes each element of given iterable.
 - iter : It is a iterable which is to be mapped.
- list() function takes an iterable construct and turns it into a list.
 - Syntax: list([iterable])

```
def doubler(i):  
    return i*i
```

```
L=[1,2,3,4,5]  
result=map(doubler,L)  
print(list(result))
```

Lambda functions

In Python, an anonymous function means that a function is without a name₁.

Similarly, the ***lambda*** keyword is used to define an anonymous function in Python. It has the following syntax:

lambda arguments: expression

- This function can have any number of arguments but only one expression, which is evaluated and returned.
- One is free to use lambda functions wherever function objects are required.
- You need to keep in your knowledge that lambda functions are syntactically restricted to a single expression.
- It has various uses in particular fields of programming besides other types of expressions in functions.

<https://www.geeksforgeeks.org/python-lambda-anonymous-functions-filter-map-reduce/>

Lambda functions

Example 1. One argument

```
Sqw= lambda i:i*i  
print(Sqw(6))
```

Example 2. Two arguments

```
PWP = lambda a,n:a**n  
print(PWP(6,3))
```

Example 3. Using with map()

```
L=[1,2,3,4,5]  
print(list(map(lambda a:a*2, L)))
```

Example 4. Using with filter()

```
my_list = [1, 5, 4, 6, 8, 11, 3, 12]  
new_list = list(filter(lambda x:  
(x%2 == 0) , my_list))  
print(new_list)
```